CSCE-685: DIRECTED STUDIES - Project Report

Name: Navya Priya Nandimandalam
UIN: 136004813

CodeChat: Conversational Repository Assistant

GitHub Repository: https://github.com/navyapriyanandi-tamu/Directed_Studies_136004813.git
Demo: https://youtu.be/KeQ7TwVmTnU

Abstract

Understanding an unfamiliar codebase is one of the slowest parts of software development, especially when the project's documentation is sparse, outdated, or scattered across files. This project introduces CodeChat, a system that brings traditional code analysis together with AI to make Python repositories easier to explore through natural conversation. Rather than stopping at syntactic structure like most existing tools, CodeChat takes a hybrid approach where Abstract Syntax Tree (AST) parsing captures the structural skeleton of the code, while Claude Haiku 4.5 provides semantic reasoning to explain what each component does and how it fits into the larger design. From this analysis, CodeChat produces both human-readable and technical summaries at the function, module, and repository levels, builds architectural and dependency diagrams, and exposes the entire repository through an interactive chat interface where developers can ask natural-language questions. To assess the quality of these outputs, we adopted an LLM-as-a-Judge evaluation strategy where summaries are produced by Claude Haiku 4.5 and then scored by a more capable model, Claude Sonnet 4.5, against criteria such as clarity, correctness, and completeness. We applied this evaluation across several Python repositories of varying size and domain. Results demonstrate that the hybrid approach produces accurate, context-aware explanations without requiring embedding or retrieval infrastructure, while keeping cost and complexity low. CodeChat shows how AI can enhance traditional code analysis to help developers onboard faster and understand unfamiliar projects with less friction.

Introduction

- **Problem Statement**

  Software repositories are growing larger and becoming more complex every year, making it harder for developers to understand, maintain, and onboard to new codebases. Research suggests that developers spend up to 60% of their time reading and reasoning about existing code rather than writing new features, and the cost of that overhead grows quickly as projects mature. This challenge is especially visible in the Python ecosystem, which is heavily used across data science, web development, machine learning, and automation, but where rapid iteration often leaves documentation incomplete, fragmented, or out of date. README files, inline comments, and manually maintained guides tend to drift away from the actual implementation as code evolves, leaving developers to reconstruct intent directly from the source.

  When approaching a new Python repository, developers typically run into three persistent challenges:

  - **Understanding the high-level architecture and purpose of the codebase:** Traditional static analysis tools like Pylint or Radon provide metrics about code complexity and style

violations but fail to convey the semantic intent or the "story" of the code. These tools can identify what the code is structurally, but not why it exists or how components work together to solve problems.

- **Navigating large numbers of files and dependencies without losing context:** Real-world repositories contain hundreds of files with intricate import chains and cross-module dependencies. Manual tracing is slow, and raw dependency graphs show that modules are connected without explaining why those connections matter.

- **Querying the codebase in a natural, conversational way:** Most existing tools only produce static output like summaries, graphs, or dashboards, that developers have to read on their own. There is no easy way to just ask "how does authentication work here?" and get a direct answer. Retrieval-based systems can do this, but they add too much cost and setup for most developers.

- **Motivation**

  Recent advances in Large Language Models (LLMs) present an opportunity to transform how developers understand code. Models like Claude Haiku 4.5 and Claude Sonnet 4.5 demonstrate strong abilities to comprehend programming languages, explain complex logic in natural language, and reason about software architecture at semantic levels that go beyond syntactic pattern matching. However, using LLMs alone for code analysis has significant limitations, they can hallucinate incorrect information when not grounded in actual code structure, lack the rigor of deterministic parsing, and may miss patterns that are obvious to traditional static analysis tools.

  This motivates a hybrid approach that combines the complementary strengths of both techniques. Abstract Syntax Tree (AST) parsing provides reliable, deterministic extraction of code structure such as function signatures, class hierarchies, import dependencies, and call relationships. This structural foundation is fast, accurate with no hallucinations, and scales efficiently to large repositories. LLMs add the critical semantic layer, they can explain why code exists, what problem it solves, and how components relate in ways that go beyond simple structural connections.

- **Objectives**

  This project develops CodeChat, an intelligent code analysis system that transforms Python repositories into interactive, conversational documentation. The system implements four primary objectives:

  - **Repository Summarization:** Generate function-level, module-level, and repository-level summaries using AST parsing combined with LLM analysis. Provide both human-readable and technical descriptions at each level. For large modules, apply chunking strategies that group functions into manageable batches so that summary quality remains consistent across the entire codebase.

  - **Architecture Mapping:** Create high-level architectural diagrams using Mermaid.js that group related modules by logical purpose rather than directory layout. Use LLM analysis on top of AST-extracted dependencies to identify semantic categories and visualize how

different architectural layers connect.

- **Conversational Querying:** Provide an interactive chat interface where developers can ask natural-language questions about the codebase - about modules, dependencies, design intent, or specific behaviors and receive grounded, context-aware answers built from the pre-generated summaries and AST facts, without relying on embeddings or retrieval pipelines.

- **LLM-as-a-Judge Evaluation:** Assess the quality of CodeChat's generated summaries using a stronger model, Claude Sonnet 4.5, as an independent judge. Score outputs on clarity, correctness, and completeness across Python repositories of varying size and domain, providing a reproducible measure of how well the hybrid pipeline performs.

- **Key Contributions**

  This project makes several contributions to automated code comprehension:

  - A practical hybrid pipeline that combines AST parsing with LLM reasoning to produce hierarchical summaries, logical module groupings, and architectural diagrams in a single, unified system, demonstrating that hybrid approaches can scale to real-world Python repositories without retrieval infrastructure.

  - A conversational interface for code understanding that allows developers to ask natural-language questions about a repository and receive grounded, context-aware answers built directly from pre-generated AST facts and multi-level summaries, removing the need for embedding stores or RAG pipelines.

  - An LLM-as-a-Judge evaluation methodology using Claude Sonnet 4.5 to score CodeChat's Haiku-generated summaries on clarity, correctness, and completeness, providing a reproducible way to assess subjective output quality across diverse codebases.

  - Empirical evidence across multiple Python repositories of varying size and domain, including requests, flask, click, and httpx, showing that the hybrid approach produces accurate, useful documentation while keeping cost and complexity low.

Related Work

CodeChat builds on recent work in LLM-based code understanding and hybrid pipelines that combine static analysis with language-model reasoning. We review two key areas.

- **LLM-Based Code Documentation**

  RepoAgent by Luo et al. (2024 - https://arxiv.org/abs/2402.16667) is an LLM-powered framework for repository-level documentation that captures cross-file dependencies and keeps documentation synchronized as code evolves. Their evaluation on Python projects showed strong qualitative results, though the system produces only static text and does not support interactive querying or architectural visualization.

Diggs et al. (2024 - https://arxiv.org/html/2411.14971v1) studied LLMs for documenting legacy languages like MUMPS and mainframe assembly. Domain-expert ratings showed LLM-generated comments matched human quality for MUMPS (GPT-4 scored 9.1/10 on factual accuracy) but performed poorly on assembly. The work also revealed that automated metrics such as ROUGE and BLEU correlate weakly with human judgment (best score: 0.34), exposing a key challenge in scaling documentation evaluation.

- **Hybrid Static Analysis with LLMs**

  Li et al. (2024 - https://dl.acm.org/doi/10.1145/3649828) combined static analysis with LLMs to detect Use-Before-Initialization bugs in the Linux kernel. Static analysis extracts path constraints, and the LLM reasons about whether those constraints can realistically be satisfied, significantly reducing false positives compared to pure static analyzers.

  Li et al. (2025 - https://arxiv.org/abs/2405.17238) proposed IRIS, a neuro-symbolic approach for security vulnerability detection that combines LLMs with static analysis to address taint analysis problems. Traditional tools like CodeQL require manually specified taint definitions for sources (functions that introduce taint), sanitizers (functions that remove it), and sinks (dangerous operations), a process that is time-consuming and requires expertise. IRIS automates this by using LLMs to infer these specifications from function names, signatures, and implementations, after which traditional static analysis performs whole-repository data-flow tracking. This division of labor pairs LLMs' semantic understanding of code purpose with static analysis's rigorous whole-program reasoning. On CWE-Bench-Java, IRIS detected 55 vulnerabilities compared to CodeQL's 27, including 4 previously unknown ones, a significant improvement over traditional static analysis

- **How CodeChat addresses these gaps**

  CodeChat makes three key advances over prior work:

  - **Interactive, Multi-Format Documentation**: Unlike RepoAgent and Diggs et al., which produce only static text, CodeChat surfaces documentation through an interactive interface. Human-readable summaries support non-technical stakeholders, technical summaries serve developers, Mermaid.js diagrams group modules by semantic purpose rather than directory structure, and a chat interface lets developers query the codebase in natural language, turning passive documentation into an explorable resource.

  - **Hybrid Analysis for Repository Comprehension**: Where Li et al. apply hybrid static-plus-LLM analysis to bug and vulnerability detection, CodeChat applies the same philosophy to understanding code. AST parsing extracts the structural backbone, and Claude Haiku 4.5 adds semantic explanations at the function, module, and repository levels, combining deterministic parsing with contextual reasoning, without embeddings or retrieval infrastructure.

  - **LLM-as-a-Judge Evaluation**: To address the evaluation gap highlighted by Diggs et al. , where ROUGE and BLEU correlate weakly with human judgment, CodeChat uses Claude Sonnet 4.5 as an independent judge to score generated summaries on clarity, correctness, and completeness, providing a scalable, reproducible alternative to manual expert review.

Methodology

- **System Overview:** CodeChat implements a multi-stage pipeline that transforms Python repositories into interactive, multi-level documentation through six core components:

    - AST Parser - Extracts functions, classes, imports, and inter-module dependencies from every Python file.
    - Function Summarizer - Generates human-readable and technical summaries for each function using Claude Haiku 4.5
    - Module Summarizer - Aggregates function summaries into module-level descriptions, with chunking support for large modules.
    - Repository Summarizer - Combines module summaries into a repository-level overview.
    - Module Grouper & Diagram Generator - Uses LLM reasoning over module summaries to cluster files into logical architectural groups, then renders Mermaid.js dependency diagrams.
    - Conversational Interface - Surfaces all generated artifacts in an interactive dashboard, including a chat panel where developers can ask natural-language questions about the codebase.

  CodeChat follows a bottom-up aggregation strategy. The AST Parser first runs over every Python file, deterministically extracting functions, classes, imports, and inter-module dependencies - this acts as the grounding layer for everything that follows. The Function Summarizer then prompts Claude Haiku 4.5 to produce two parallel descriptions per function: a plain-language summary for general readers, and a technical summary for developers. These roll up into module-level summaries, and modules roll up into a single repository-level overview, keeping explanations consistent across granularities.

  In parallel, the Module Grouper feeds the module summaries back to the LLM to organize files into logical architectural categories based on purpose rather than directory layout. The Diagram Generator combines these groupings with AST-derived import edges to render Mermaid.js diagrams. All intermediate artifacts, AST output, function/module/repo summaries, and logical groups, are written to a single context_<repo>.json file, which makes the pipeline reproducible and acts as input for the evaluation step. The Streamlit dashboard reads from this file and exposes the summaries, diagrams, and a chat panel where developers can ask natural-language questions about the codebase. Figure 1 illustrates the full pipeline and data flow.

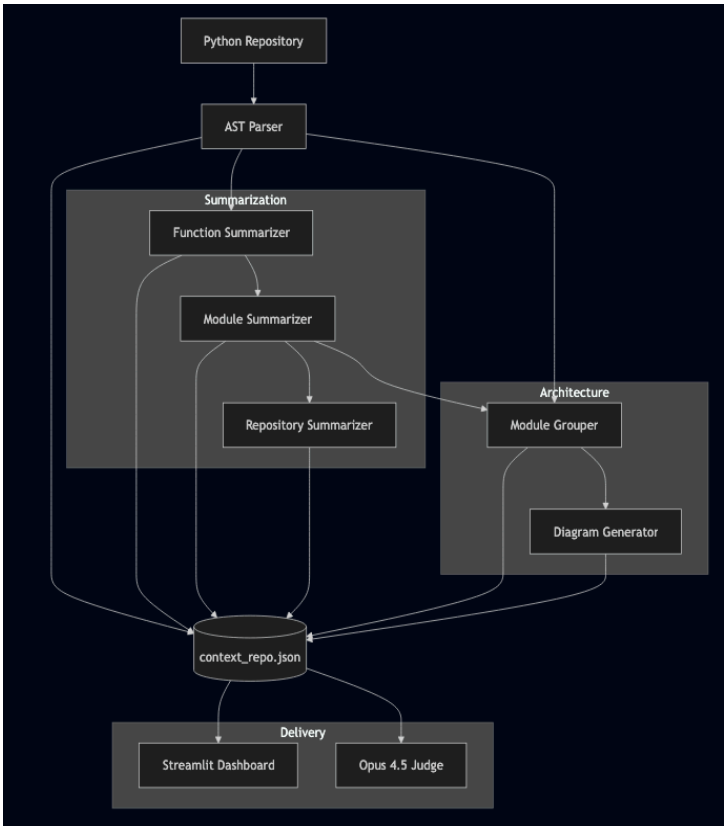


Figure 1: CodeChat system pipeline

- **AST Parser:** The AST Parser is the first stage of the pipeline, responsible for turning raw Python source into a structured, machine-readable representation of the project. It builds on Python's built-in ast module, parsing each file statically, no imports are resolved, no code is executed, and no dependencies need to be installed, which keeps the analysis safe even on unfamiliar repositories. For every Python file, the parser extracts function signatures (with argument types, return annotations, decorators, line counts, and full source segments), class declarations with their methods and base classes, all import and from ... import statements, top-level variable assignments, and an entry-point flag for files containing if __name__ == "__main__":. It also walks each function body to record the names it references and tracks module-level function calls and class instantiations separately, producing the dependency information used by later stages.

  To avoid wasting LLM calls on trivial code, each function is tagged with a needs_llm_summary flag - set to False for stubs under three lines. The parser's output is a single dictionary keyed by relative file path, containing per-file metadata, call traces, and an aggregate __analysis_summary__ block, the structural backbone that every later stage builds on.

- **Repository Summarization Strategy:** The summarization pipeline in CodeChat is layered into three levels i.e., function, module, and repository, and each layer is constructed from the layer beneath it. Function summaries roll up into module summaries, and module summaries roll up into the repository summary. This bottom-up flow is deliberate, by giving every higher level a concrete set of lower-level inputs to work from, the LLM is rarely asked to reason about anything it has not already seen described in detail and hence avoids hallucinations.

- **Function Summarizer:** At the bottom of the hierarchy, the Function Summarizer produces two parallel descriptions for each function: a plain-language human summary for readers new to the code, and a technical summary that retains the details a developer would need for maintenance. Both come from the same prompt, letting a single function's documentation serve both onboarding and engineering use. The prompt itself is grounded in AST-extracted facts - the full source text, signature with type hints, decorators, docstring, line count, and the names called inside the body, so the model has concrete structural context to anchor its explanations. Each summary is capped at 5-6 lines, which is the tightest of the three levels in the pipeline (modules use 6-8 lines, repositories 8-10), reflecting the fact that function-level documentation is meant to stay compact and pointed rather than expansive. Trivial functions flagged earlier by the parser (fewer than three lines) are skipped to avoid wasting calls.

  When a function is unusually large, roughly 200 lines or 12,000 characters and above, the summarizer switches to a chunking strategy. The body is split into 60-line segments with 15-line overlap, so statements at the boundaries stay interpretable. Each chunk is summarized individually with shared context about the overall signature, and the partial summaries are merged into one coherent function-level description. This keeps even long, dense functions within the model's effective context window while still producing a complete account of their behavior. Figure 2 below shows the workflow.

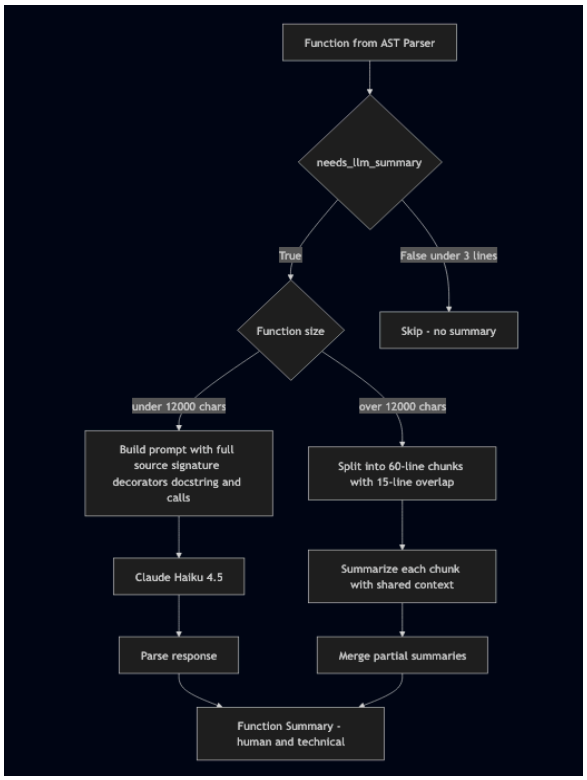


Figure 2: Function summarization workflow

- **Module Summarizer:** Sitting one level up from the Function Summarizer, the Module Summarizer produces a description for each Python file in the repository. It takes in the function summaries from the previous stage along with AST-extracted metadata - class

definitions with methods and base classes, import statements with aliases and dependencies, and per-function metadata such as signatures, decorators, and line counts. From this, it generates both a human and a technical summary that capture the module's architectural role, its main exports, the design patterns it relies on, and its dependencies on other modules. The prompt caps both summaries at 6-8 lines each, slightly longer than the function-level limit of 5-6 lines, giving the LLM enough room to describe the module's role and dependencies while keeping each summary compact enough to scan quickly.

For files with more than 30 functions, the summarizer applies a chunked aggregation strategy: function summaries are split into groups of 30, an intermediate summary is produced for each group, and these are then merged into a final module-level summary. This keeps coherence intact even on arbitrarily large files. The value of this middle layer is that it surfaces cohesion and coupling patterns invisible at the function level - a file containing routines for tokenization, syntax validation, and AST construction, for instance, is described as a "Syntax Analysis Module" rather than as a flat list of functions, helping developers grasp module boundaries without reading every function individually. Figure 3 below shows the workflow.

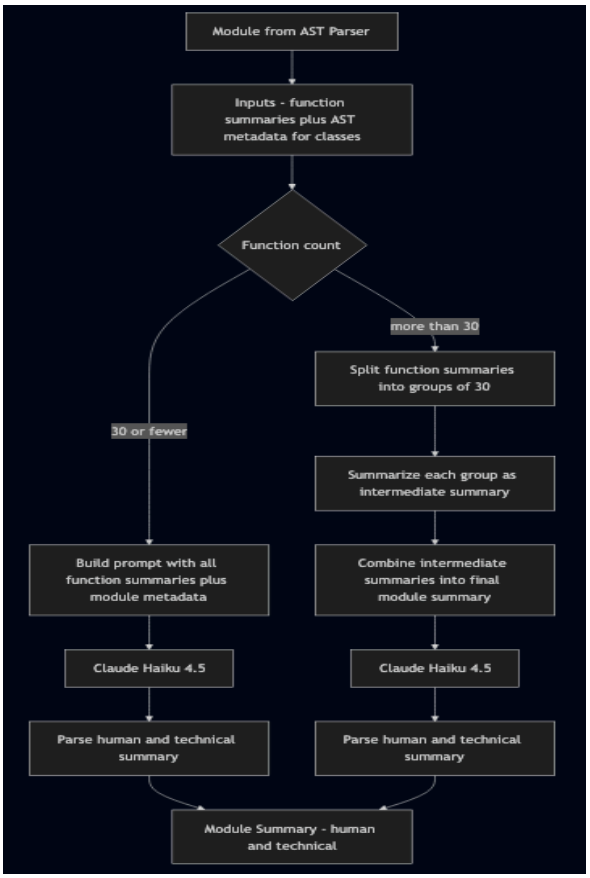


Figure 3: Module Summarization workflow

- **Repository Summarizer:** At the top of the hierarchy, the Repository Summarizer combines all module summaries with the AST-derived statistics produced earlier - total module, function, and class counts into a single repository-level overview that describes

what the project is, how it is organized, and which technologies it uses. The prompt caps the human and technical sections at 8-10 lines each, slightly more expansive than the lower layers (5-6 lines for functions, 6-8 lines for modules), giving the LLM enough room to describe the overall architecture and tech stack while keeping the output concise enough to act as an at-a-glance project map. For projects with more than 40 modules, the summarizer switches to a two-stage approach: modules are first grouped by directory (e.g. src/core, src/api, tests), each group is summarized into a description of its collective purpose, and the group summaries are then merged into the final repository overview. The 40-module threshold is chosen to give the LLM enough context for a coherent global view while staying within a single API call's token budget.

Because the summary is built from previously generated module summaries rather than from raw source code, it reflects what the project actually implements rather than what the LLM might infer or fabricate from filenames alone. Figure 4 below shows the workflow.

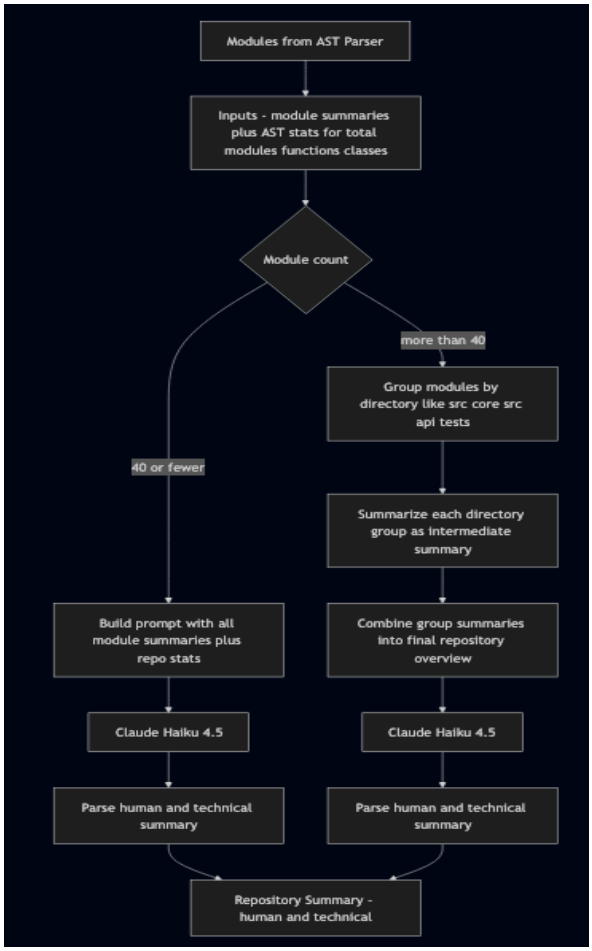


Figure 4: Repository Summarization workflow

- **Architecture Visualization:** CodeChat produces its architecture diagrams in two stages, handled by two separate components - the Module Grouper and the Diagram Generator.

  Module Grouper - Rather than relying on directory layout (which is often flat, inconsistent, or organized by file type instead of function), the Module Grouper asks Claude Haiku 4.5 to read the

module summaries from the previous stage and cluster the modules by their logical purpose. The prompt provides each module's human and technical summary along with its file path, and the model returns a JSON mapping of category names to module lists. For example, files like auth.py, session.py, and permissions.py get grouped under Security Layer, or parser.py, lexer.py, and tokens.py under Language Processing Core - groupings that reflect how a developer would mentally organize the codebase, even if its directories don't. If the LLM call fails or returns malformed JSON, the system falls back to deterministic directory-based grouping.

Diagram Generator - Given the logical groups and the import edges extracted earlier by the AST Parser, the Diagram Generator produces two views of the architecture: a plain-text ASCII dependency tree for terminal inspection, and a Mermaid.js flowchart for documentation. The Mermaid view places each logical group inside its own subgraph, draws aggregated import arrows between groups rather than between individual files, and applies a semantic color scheme based on group-name keywords - blue for entry points, green for core and API components, purple for models, orange for network adapters, red for security, yellow for utilities, gray for config, teal for plugins and so on. The group-level rendering keeps the diagram readable regardless of project size and makes architectural layers visible at a glance - for example, surfacing patterns like "UI depends on Business Logic depends on Data Access." Output is written as a .mmd file that renders natively in GitHub, VS Code, and most documentation platforms. Figure 5 below shows the workflow

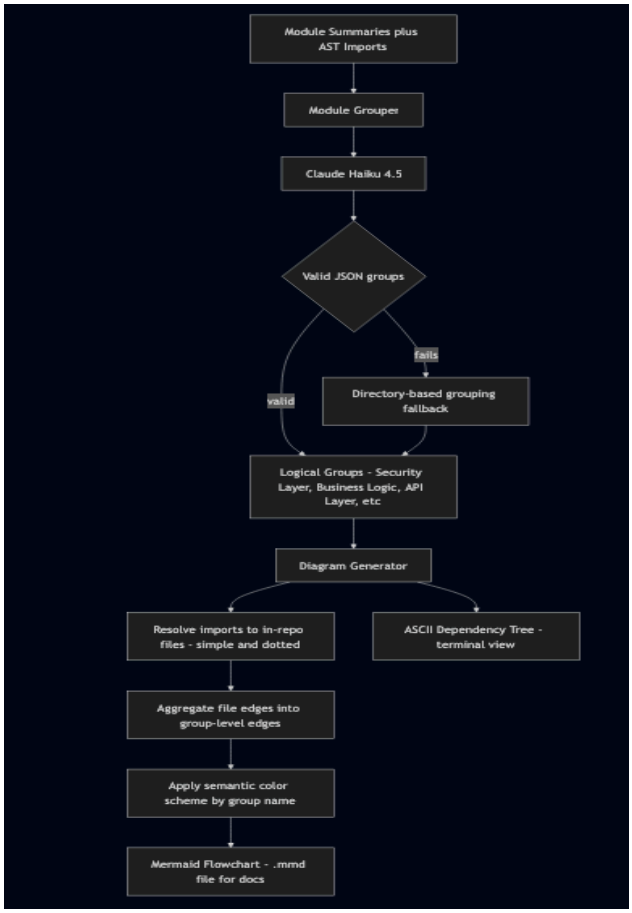


Figure 5: Architecture Visualization workflow

- **Conversational Querying:** CodeChat's chat interface is intentionally lightweight, it relies entirely on the structured artifacts already produced by the rest of the pipeline, so no embedding store, no vector index, and no retrieval-augmented-generation layer is needed. When a developer enters a question, the system loads the repository's pre-generated context_<repo>.json and assembles a prompt that combines the user's query with a flattened view of the analysis: the repository name, the AST-derived statistics (total modules, functions, classes), the repository-level human and technical summaries, every module's human summary keyed by file path, and every function's human summary keyed by module::function. This complete view of the repository is sent to Claude Haiku 4.5 with simple instructions to answer concisely in Markdown, and the response is streamed back into the Streamlit chat panel.

  Because every level of the hierarchy is present in the prompt, the model can answer questions at any granularity without an explicit retrieval step - high-level "what does this project do?" questions are grounded in the repo and module summaries, while targeted questions like "how does authentication flow?" or "which module handles request retries?" are grounded in the function-level summaries. This works because the summaries themselves are already short and structured: feeding all of them at once stays within Haiku's context window even on medium-sized repositories like flask or httpx.

Implementation

- **System Implementation**
  - **Technology Stack:** The implementation relies on a small, focused set of libraries:
    - Python ast module: Built-in library for parsing Python source code into abstract syntax trees, used as the deterministic structural analysis layer.
    - TAMU AI Chat API (OpenAI-compatible): Acts as a proxy to Claude models hosted on AWS Bedrock. Accessed through the openai Python SDK with a custom base_url.
    - Claude Haiku 4.5 (protected.Claude-Haiku-4.5): Generates all function, module, and repository summaries as well as the logical groupings used in architecture diagrams. It is also used for conversational chat.
    - Claude Sonnet 4.5 (protected.Claude Sonnet 4.5): Used as an independent judge in the LLM-as-a-Judge evaluation pipeline.
    - Streamlit: Powers the interactive web dashboard that surfaces summaries, diagrams, and the conversational chat interface in a single browser-based UI.
    - JSON: Used as the storage format for context_<repo>.json and evaluation_<repo>.json, keeping all intermediate artifacts (AST output, summaries, logical groups, evaluation scores) human-readable and reproducible.

- **Project Structure:**

```
codechat/
    ├── src/
    │   ├── ast_parser.py           # AST analysis: functions, classes, imports, calls
    │   ├── function_summarizer.py  # Function-level summarization with chunking
    │   ├── module_summarizer.py    # Module-level aggregation
    │   ├── repo_summarizer.py      # Repository-level overview
    │   ├── module_grouper.py       # LLM-driven logical grouping with directory fallback
    │   └── diagram_generator.py    # ASCII + Mermaid.js visualization
    ├── repos/                      # Cloned target repositories under analysis
    ├── diagrams/                   # Generated ASCII trees and .mmd files
    ├── config.py                   # API keys, model selection, skip patterns
    ├── analyze_any_repo.py         # Main pipeline orchestrator (Haiku 4.5)
    ├── streamlit_app.py            # Web UI dashboard with chat interface
    ├── evaluate_with_sonnet.py     # LLM-as-a-Judge evaluation pipeline (Sonnet 4.5)
    ├── codechat.py                 # CLI entry point
    ├── requirements.txt            # Python dependencies
    ├── context_<repo>.json         # Full analysis state per repository
    └── evaluation_<repo>.json      # Sonnet judge scores per repository
```

- **Configuration Management:** The config.py module reads system-wide settings from the shell environment (with optional support for a local .env file via python-dotenv) and exposes them to every component in the pipeline:

  - TAMUS_AI_CHAT_API_KEY: Authentication token for the TAMU AI Chat API
  - TAMUS_AI_CHAT_API_ENDPOINT: Base URL for the TAMU API (defaults to https://chat-api.tamu.ai).
  - LLM_MODEL: Model alias used for all generation tasks - function, module, and repository summaries, plus logical module grouping (defaults to protected.Claude-Haiku-4.5).
  - JUDGE_MODEL: Model alias used by the evaluation pipeline as an independent judge (defaults to protected.Claude Sonnet 4.5).
  - SKIP_PATTERNS: Directories excluded during repository ingestion - ['test', 'tests', 'venv', '__pycache__', '.git', 'node_modules', '.pytest_cache'], so that analysis focuses on production code rather than test fixtures or vendored files.

- **Pipeline Orchestration (codechat.py):** End-to-end execution is driven by analyze_any_repo.py, which runs each component in sequence and feeds every stage's output into the next. The AST Parser walks every Python file in the target directory and produces a structured representation of its functions, classes, imports, and calls. That output drives the Function Summarizer, which generates paired human and technical descriptions for each function. Those function summaries feed the Module Summarizer, and module summaries in turn feed the Repository Summarizer for the project-level overview. The Module Grouper then sends module summaries back to the LLM for logical clustering, and the Diagram Generator combines those clusters with the AST import graph to emit an ASCII tree and a Mermaid .mmd file. Every artifact - AST output, summaries at all three levels, and logical groups, is consolidated into one context_<repo>

.json file, which becomes the shared input for the Streamlit dashboard and the evaluate_with_sonnet.py evaluation pass.

For convenience, codechat.py chains the pipeline together with the dashboard: a single command (python3 codechat.py <repo_path>) runs the analysis and launches Streamlit with the new repository pre-selected. A --skip-analysis flag opens the dashboard against an existing context file without rerunning the pipeline.

- **Context Persistence:** CodeChat uses a single JSON file per repository, context_<repo>.json, as the shared store for every artifact produced by the pipeline. The orchestrator builds an in-memory context dictionary as each stage completes - AST output, function summaries, module summaries, repository overview, and logical groups and writes the full dictionary to disk at the end of the run. The same file is then read back by the Streamlit dashboard for browsing and chat, and by the evaluation script (evaluate_with_sonnet.py) for the LLM-as-a-Judge pass. This single-file approach keeps state inspectable and reproducible, and lets each repository be re-analyzed or re-evaluated independently. The context file follows a flat, hierarchical structure that mirrors the pipeline (Figure 6):

```
{
   "repo_name": "<repository-name>",
   "ast_results": { /* per-file AST metadata: functions, classes, imports, calls */ },
   "function_summaries": { /* per-module {function_name: {human, technical}} */ },
   "module_summaries": { /* per-module {human, technical} */ },
   "repo_summary": { "human": "...", "technical": "..." },
   "logical_groups": { /* {category_name: [file_paths]} */ }
 }
```

```
  repo_name:          "httpx"
▶ ast_results:        { "httpx/__init__.py": {…}, "httpx/__version__.py": {…}, "httpx/_api.py": {…}, … }
▶ function_summaries: { "httpx/__init__.py": {…}, "httpx/_api.py": {…}, "httpx/_auth.py": {…}, … }
▶ module_summaries:   { "httpx/__init__.py": {…}, "httpx/__version__.py": {…}, "httpx/_api.py": {…}, … }
▶ repo_summary:       { human: "HTTPX is a modern Python HTTP client library that enables developers to easily make web requests to APIs and servers. It pro…al for building
                      scripts, and testing web applications without needing a live server.", technical: "HTTPX is a modern async-first HTTP client built on httpcore for low-
                      Architecture co…TTPS/proxies (HTTP/HTTPS/SOCKS5), streaming uploads/downloads, and multipart form data with a unified sync/async interface." }
▶ logical_groups:     { "Application Entry": (3)[…], "API Layer": (2)[…], "Authentication & Security": (1)[…], … }
```

Figure 6: CodeChat context structure

- **AST Parser Implementation (ast_parser.py):**

  - **File Discovery and Parsing**: The CodeAnalyzer class scans the target directory with Path.rglob("*.py") and filters the results against config.SKIP_PATTERNS (test, tests, venv, __pycache__, .git, node_modules, .pytest_cache), so test fixtures, build caches, and VCS internals never reach the parser. Surviving files are passed through ast.parse() to produce an AST without ever executing the code. Per-file parse failures are caught and logged, letting the rest of the repository continue uninterrupted.

  - **Metadata Extraction**: Each AST is then traversed with ast.walk() to collect:
    - Functions: captured from FunctionDef and AsyncFunctionDef nodes along with argument types, return annotations, decorators, line counts, an is_async flag, and the original source text via ast.get_source_segment(). Each function also

receives a needs_llm_summary flag set to True only when its body is at least three lines, so trivial stubs can be skipped during summarization.

- Classes: pulled from ClassDef nodes with method lists, base classes, and docstrings.
- Imports: both import and from … import forms, recorded with module paths, aliases, and line numbers.
- Module-level variables: top-level Assign nodes, along with what they are assigned to.
- Entry-point markers: the presence of if __name__ == "__main__": and the function calls made inside it.

- **Call and Dependency Tracking**: For every function, the parser recursively records every name referenced inside the body such as call targets, attribute accesses, names passed as arguments, and assignment sources, yielding a per-function calls list. Calls and class instantiations that occur outside any function or class (top-level setup code) are recorded separately at the module level. Together these feed the dependency edges used later by the Diagram Generator.

- **Output:** Once every file is processed, the analyzer returns a single dictionary keyed by relative file path, with one entry per module plus a synthetic __analysis_summary__ block holding the project-wide totals (total_modules, total_functions, total_classes) as shown in Figure 7 below. This dictionary becomes ast_results inside the context file and is the structural foundation that every later stage of the pipeline i.e., function summarization, module summarization, logical grouping, and diagram generation, builds on top of.

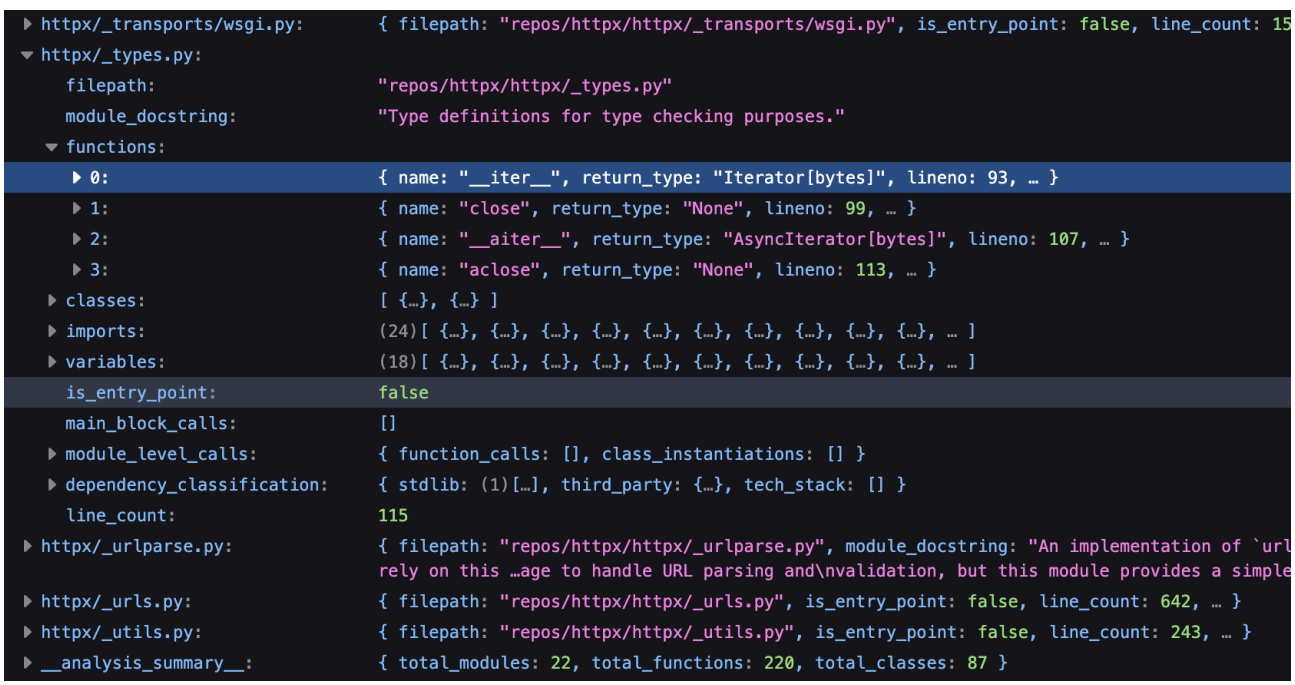


Figure 7: AST Output

- **Hierarchical Summarization Implementation**

  - **Function Summarizer Implementation(function_summarizer.py):** The FunctionSummarizer class routes each function based on size. The entry point compares the length of the function's source text against MAX_FUNCTION_CHARS = 12000 (~200 lines) and dispatches to either _summarize_full_function (single-pass) or _summarize_with_chunking. Trivial functions previously flagged by the AST Parser as needs_llm_summary=False are skipped before this check.

    In the single-pass path, one prompt is built containing the function's source, signature with type hints, decorators, docstring, line count, and the names called inside its body. A single Claude Haiku 4.5 call (max_tokens=400, temperature=1) returns a two-section response capped at 5 - 6 lines each. The chunked path splits the body via code.split('\n') into 60-line segments with 15-line overlap (CHUNK_LINES = 60, CHUNK_OVERLAP = 15), summarizes each chunk with shared context about the overall signature and variables, and then runs a final aggregation call that merges the partial summaries into one coherent function-level description.

    The model's reply is parsed by _parse_two_level_summary(), which scans for the **Human Summary:** and **Technical Summary:** headers and accumulates the lines that follow each into the corresponding field as seen in Figure 8. The output is a {"human": "...", "technical": "..."} dict attached to the function name and stored in the orchestrator's in-memory context, which is later persisted to context_<repo > .json.

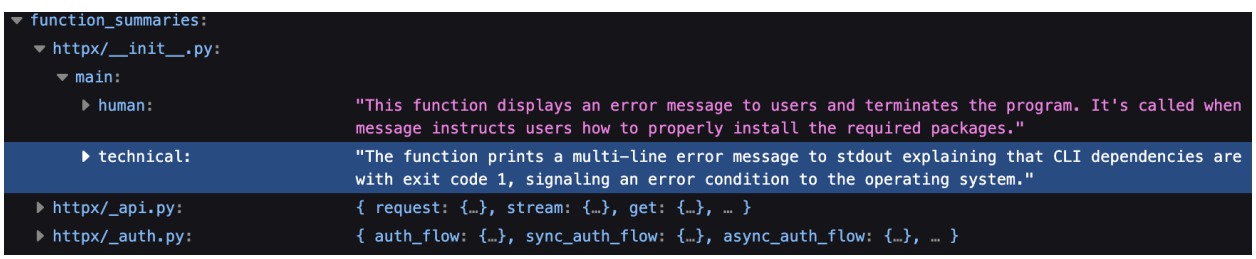


Figure 8: Function summaries generated

  - **Module Summarizer Implementation(module_summarizer.py):** The ModuleSummarizer class takes the nested function-summary dict ({module_path: {func_name: {"human": ..., "technical": ...}}}) along with AST metadata for each module, and routes by size. The entry point compares the count of function summaries against MAX_FUNCTIONS_PER_CALL = 30 and dispatches to either a direct path (_build_module_prompt) or a chunked path (_summarize_large_module).

    In the direct path, the prompt is assembled from the module path, its docstring, class definitions formatted as class ClassName(BaseClass) with method names, the flat list of imports, and all function summaries inline. A single Claude Haiku 4.5 call (max_tokens=500, temperature=1) produces the dual human / technical output capped at 6–8 lines each.

    For modules with more than 30 functions, the chunked path partitions the function summaries into groups of 30, summarizes each group via a separate call, and then runs

a final aggregation call that combines the group summaries with the module's metadata into one coherent module-level description. Modules with no functions at all (config files etc.,) fall back to _summarize_module_from_code(), which produces a minimal summary from the module docstring and top-level variable definitions alone.

Response parsing reuses the same _parse_two_level_summary() helper as the function stage, scanning for the **Human Summary:** and **Technical Summary:** headers and accumulating the lines that follow into the matching fields as seen in Figure 9. The resulting {"human": "...", "technical": "..."} dict is stored under the module path in the in-memory context, which the orchestrator later persists to context_<repo > .json.

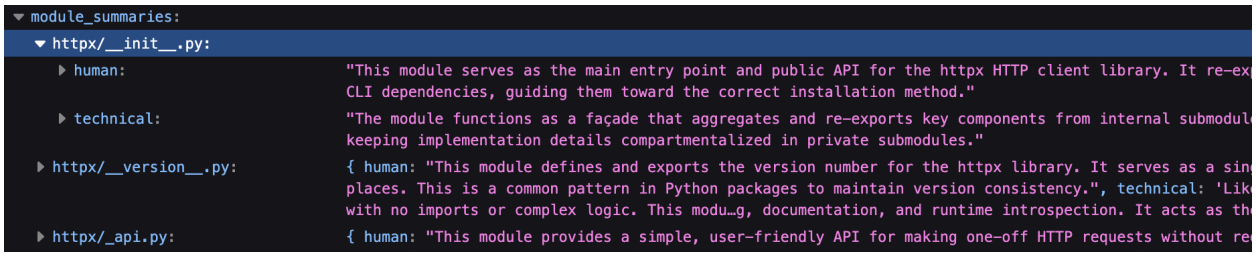


Figure 9: Module summaries generated

- **Repository Summarizer Implementation(repo_summarizer.py):** The RepoSummarizer class receives the module summaries ({module_path: {"human": ..., "technical": ...}}) along with the AST-derived statistics such as total modules, functions, and classes and routes by size. The entry point compares the module count against MAX_MODULES_PER_CALL = 40 and dispatches to either a direct path (_build_repo_context) or a chunked path (_summarize_large_repo).

  In the direct path, the prompt is built by concatenating every module's human and technical summary alongside the repository-wide statistics, and a single Claude Haiku 4.5 call (max_tokens=600, temperature=1) produces the dual human / technical output capped at 8–10 lines each, slightly longer than the function and module-level limits to leave room for an architectural narrative.

  For repositories with more than 40 modules, the chunked path groups module summaries by their parent directory using pathlib.Path(module_path).parent, generates one intermediate summary per directory group via a separate call, and then runs a final aggregation call that merges those group summaries with the repository statistics into the final repository overview. The 40-module threshold is chosen to keep the direct-path prompt comfortably inside Haiku's effective context window while still giving it enough material to describe the entire system at once.

  Response parsing reuses the same _parse_two_level_summary() helper used by the function and module stages, scanning the model's output for the **Human Summary:** and **Technical Summary:** headers and accumulating the following lines into the matching fields as shown in Figure 10 below. The resulting {"human": "...", "technical": "..."} dict is stored under the repo_summary key in the in-memory context, which is then persisted to context_<repo > .json.

```
▼ repo_summary:
  ▶ human:        "HTTPX is a modern Python HTTP client library that enables developers to easily
                  and testing web applications without needing a live server."
  ▶ technical:    "HTTPX is a modern async-first HTTP client built on httpcore for low-level netw
                  multipart form data with a unified sync/async interface."
```

Figure 10: Repository Summary generated

- **Architecture Visualization Implementation**

  - **Module Grouper(module_grouper.py):** The ModuleGrouper class accepts the module-summary dict produced earlier and builds a single prompt that lists each module path next to its human summary. The prompt asks Claude Haiku 4.5 to return a JSON object whose keys are architectural category names (suggested archetypes include Application Entry, Business Logic, Data Storage, API Layer, Authentication & Security, Utilities) and whose values are arrays of module paths as seen in Figure 11 below.

    The call uses max_tokens=2000 to leave room for grouping responses on repositories with many modules. The response is then handled by _parse_response, which strips markdown code fences if present, parses the JSON, and validates that every input module appears in exactly one group, any modules missing from the LLM's output are automatically routed into an "Other" category so the downstream diagram is never incomplete.

    If the call raises an exception or the JSON cannot be parsed, the implementation falls back to _fallback_grouping, which extracts the first directory component from each module path (module_path.split('/')[0]) and groups files accordingly, with files at the repository root collected under "Root Level".

  - **Diagram Generator(diagram_generator.py):** The DiagramGenerator class consumes the logical groups along with the AST results and produces two artifacts. Dependency edges are first built by _extract_dependencies, which constructs a lookup table mapping module stems, full paths without .py, and dotted-path variants to actual repository files, each module's imports are then resolved by checking this table, so simple imports like import model match through stem lookup and dotted imports like from task_manager.task_handler import X resolve by converting dots to slashes. The result is a per-file dependency map of in-repository imports, ignoring third-party and standard-library names.

    The first output is a plain-text ASCII dependency tree generated by _generate_ascii_dependencies, which iterates modules alphabetically, prints each module followed by its indented import list, and uses Unicode box-drawing characters to render the tree visually.

    The second output is a Mermaid flowchart produced unconditionally by _generate_simplified_mermaid even on small repos, the diagram is rendered at the group level rather than the file level so it stays readable as the codebase grows. File-level dependencies are aggregated into group-to-group edges by checking which group contains each dependent file. A get_group_color() helper performs lowercase keyword matching on group names to assign hex color codes, blue for entry points, green for core

and APIs, purple for models, orange for network adapters, red for security, yellow for utilities, gray for configuration, teal for plugins, and a neutral gray default for anything that doesn't match, so readers can identify functional layers visually without reading every label. The generated graph TD syntax wraps each group in a subgraph, applies the assigned color via a style directive, and connects groups with Group1 --> Group2 arrows. Both files are written to the diagrams/ directory via _save_diagram(), with dependencies_<repo>.txt for the ASCII tree and mermaid_<repo>.mmd for the Mermaid flowchart, ready to render in GitHub, VS Code, or any other Mermaid-aware tool.

```
▼ logical_groups:
  ▶ Application Entry:          (3)[ "httpx/__init__.py", "httpx/__version__.py", "httpx/_main.py" ]
  ▶ API Layer:                  [ "httpx/_api.py", "httpx/_client.py" ]
  ▶ Authentication & Security:  [ "httpx/_auth.py" ]
  ▶ Configuration:              [ "httpx/_config.py" ]
  ▶ Models:                     [ "httpx/_models.py", "httpx/_status_codes.py" ]
  ▶ Data Processing:            (5)[ "httpx/_content.py", "httpx/_decoders.py", "httpx/_multipart.py", "ht
  ▶ Transport Layer:            (6)[ "httpx/_transports/__init__.py", "httpx/_transports/base.py", "httpx/
                                mock.py" ]
  ▶ Error Handling:             [ "httpx/_exceptions.py" ]
  ▶ Utilities:                  [ "httpx/_types.py", "httpx/_utils.py" ]
```

Figure 11: Logical groups generated by Haiku

- **Conversational Querying Implementation:** The chat layer loads a single context_<repo>.json file and assembles its contents into a flat prompt via build_context_summary() including repository name, AST statistics, the repository summary, every module's human summary keyed by path, and every function's human summary keyed by module::function. This string is wrapped in a fixed instruction telling Claude Haiku 4.5 to answer the user's question concisely in Markdown, and sent through config.llm_call() with max_tokens=2000 and temperature=1. The streamed response is rendered directly into the active chat bubble.

  Conversation history is kept in st.session_state["messages"] so prior turns persist within a session. The sidebar discovers every context*.json in the working directory at startup, lets the user switch repositories on the fly, and exposes separate tabs for browsing the repository overview, module summaries, function summaries, architecture diagrams, and dependency graphs alongside the chat panel.

  Because every layer of the summary hierarchy is included in the prompt up front, the LLM has direct access to repo-, module-, and function-level grounding for any question; no retrieval, embedding, or vector-store component is required. The trade-off is a slightly larger per-query token bill in exchange for a system that is far simpler to deploy and reproduce.

- **Web UI (streamlit_app.py):** The Streamlit dashboard provides an interactive front-end for browsing analysis results. Repository discovery happens at startup: Path('.').glob('context_*.json') enumerates every available analysis, extracts the repo name from each filename, and offers them as a sidebar dropdown via st.selectbox. When a repository is chosen, the corresponding context file is loaded and the user can navigate between six pages from a sidebar radio menu, Chat, Overview, Architecture, Dependencies, Functions, and Modules.

- Overview (display_overview): Shows three top-line metrics (file, function, and class counts) using st.columns(3) and renders the repository summary, with the technical version tucked behind an st.expander.
- Modules (display_modules): Lists each module as an expandable card with its human summary visible by default and the technical summary inside a nested expander. A case-insensitive st.text_input search box filters the list as the user types.
- Functions (display_functions): Same pattern as the module page but grouped by module, with each function's name, human summary, and technical expander shown inside the parent module's expandable section.
- Architecture (display_architecture): Reads diagrams/mermaid_<repo>.mmd, encodes the contents into a base64 URL fragment, and links out to the Mermaid Live Editor for interactive pan/zoom, the raw Mermaid source is also rendered in a code block. The logical groups are displayed beneath as expandable sections listing the files that belong to each group.
- Dependencies (display_dependencies): Loads diagrams/dependencies_<repo>.txt and renders the ASCII dependency tree in a st.code block.
- Chat (chat_interface + handle_query): Builds a flattened textual context via build_context_summary() and sends it with the user's question to Claude Haiku 4.5. Conversation history is stored in st.session_state["messages"] and rendered as alternating user/assistant bubbles via st.chat_message.

Experimental Setup

- **Test Repositories:** We evaluated CodeChat on six Python repositories that vary in size, domain, and architectural style, in order to test the system across small educational projects as well as large production libraries. Some repositories such as test_project and gnn_training-master are small, easy-to-reason-about codebases that act as sanity checks for the pipeline, while the rest are widely used open-source libraries with well-documented public APIs.

Repository	Domain	Modules	Functions	Classes	Description
test_project	Educational	6	35	4	Small sample project used for early pipeline testing
gnn_training-master	Machine Learning	5	4	2	Graph neural network training framework
requests	HTTP Client	20	185	45	Popular Python HTTP library for interacting with web APIs
httpx	HTTP Client	22	220	87	Async-capable HTTP client built on top of HTTP/1.1 and HTTP/2
flask	Web framework	34	281	52	Lightweight WSGI web framework for Python
click	CLI Framework	31	347	78	Command-line interface toolkit built on top of decorators

The selected repositories span from 5 modules and 4 functions (gnn_training-master) at the lower end to 34 modules and 347 functions (click) at the upper end, giving the evaluation reasonable coverage across small, medium, and larger codebases. Including two HTTP-client libraries (requests and httpx) also lets us compare CodeChat's output on functionally similar projects with very different internal designs, requests follows a synchronous, classical layout while httpx is built around async transports, which provides a useful sanity check on whether the LLM is reproducing surface keywords versus genuinely capturing architectural intent.

- **Evaluation Methodology:** We evaluated CodeChat in two ways:
    - **Performance Metrics (Quantitative Metrics):** Each repository was run end-to-end through the pipeline (AST parsing → function/module/repository summarization → logical grouping → diagram generation) on a MacBook Pro running macOS 26. All generations used Claude Haiku 4.5 via the TAMU AI Chat API. We recorded analysis completion time across all six repositories.
    - **LLM-as-a-Judge Quality Evaluation:** Quality scoring is automated through evaluate_with_sonnet.py, which loads each repository's context_<repo>.json and uses Claude Sonnet 4.5 as an independent judge. The judge scores four components: the repository summary (accuracy, completeness, clarity, usefulness), every module summary (accuracy, completeness, clarity), a stratified random sample of function summaries (factual accuracy, completeness, clarity, technical depth), and the architecture diagram (completeness, logical grouping, diagram quality). To keep evaluation cost within our daily API budget especially in the case of large repositories, the function-level evaluation is capped at 50 sampled functions per repository, drawn proportionally from each module: special-method functions (Python "dunder" methods such as __init__ and __repr__) and very short functions (under five lines, where there is little substance to summarize) are filtered out, and longer functions are weighted preferentially when sampling. Each prompt requests a JSON response with 1–5 ratings, and results are written to evaluation_<repo>.json along with per-component qualitative notes, error lists, and an aggregated overall_metrics block, giving a reproducible quality signal without requiring manual review.
- **Metrics:** We measured CodeChat across two categories:
    - **Quantitative Metrics:**
        - Analysis completion time per repository
    - **Quality Metrics (via Claude Sonnet 4.5 evaluation):** For every quality assessment, the judge receives the same inputs that were originally given to Haiku 4.5 alongside Haiku's generated output, enabling independent scoring rather than self-judgment.
        - **Function / Module / Repository Summaries:**
            - Inputs: Original function source code (or AST metadata + child summaries for modules and repositories) + Haiku-generated human and technical summaries
            - Evaluation: Rated on a 1–5 scale for Factual Accuracy, Completeness, Clarity, Technical Depth (functions only), and Usefulness (repository only)

            - Outputs: Per-criterion ratings, an overall score, error lists, missing elements, and recommendations

        - **Architecture Diagram:**
            - Inputs: List of repository files + generated Mermaid source + logical groups dict
            - Evaluation: 1–5 scale on Completeness (all major modules included), Logical Grouping (semantic coherence), Diagram Quality (structure, clutter)
            - Outputs: Per-criterion ratings, an overall score, and improvement suggestions

    The four overall scores are then combined into a single weighted Overall Quality score (function summaries 35%, module summaries 25%, repository summary 20%, architecture 20%), giving a single comparable number per repository while preserving per-component detail.

- **Configuration:** All experiments used a consistent configuration:

    - **Generation (CodeChat pipeline):**
        - LLM Model: Claude Haiku 4.5 (protected.Claude-Haiku-4.5) accessed through the TAMU AI Chat API
        - Temperature: 1
        - Max Tokens: 400 (function summaries), 500 (module summaries), 600 (repository summary), 2000 (module grouping)
        - Chunking Thresholds: 200 lines / 12,000 characters (functions), 30 functions per module (module summarizer), 40 modules per repository (repository summarizer)
        - Skip Patterns: ['test', 'tests', 'venv', '__pycache__', '.git', 'node_modules', '.pytest_cache']

    - **Evaluation (LLM-as-a-Judge):**
        - LLM Model: Claude Sonnet 4.5 (protected.Claude Sonnet 4.5) - chosen as a more capable judge than the Haiku 4.5 generator
        - Temperature: 1
        - Max Tokens: 1500–3000 depending on evaluation type (1500 per module, 2000 per function, 3000 for the repository and architecture evaluations)
        - Function Sampling: stratified random sample of up to 50 functions per repository, drawn proportionally across modules, dunder methods and functions under 5 lines are filtered out, longer functions are weighted preferentially, and the random seed is fixed at 42 for reproducibility

Using a stronger, distinct model (Sonnet 4.5) as the judge over a smaller generator (Haiku 4.5) follows the standard LLM-as-a-Judge methodology, where the evaluator is independent of the production model. All artifacts are persisted as JSON: each repository's analysis is written to context_<repo>.json, and each evaluation pass is written to evaluation_<repo>.json as seen in Figure 12 below, making both generation and evaluation reproducible and easily inspectable.

```
repo_name:                      "httpx"
judge_model:                    "protected.Claude Sonnet 4.5"
generator_model:                "Claude Haiku 4.5"
▶ repo_summary_evaluation:      { overall_score: 4, accuracy: {…}, completeness: {…}, … }
▶ module_summaries_evaluation:  { average_score: 4.52, modules_evaluated: (23)[…], individual_ev
▶ function_summaries:           { "httpx/__init__.py::main": {…}, "httpx/_api.py::request": {…},
▼ architecture_evaluation:
  ▶ completeness:               { rating: 5, files_in_repo: 23, notes: "All 23 files are properl
  ▶ logical_grouping:           { rating: 5, notes: "The groups are semantically coherent and fo
                                  authentication, models, error handling, and utilities. The separ
  ▶ diagram_quality:            { rating: 2, notes: "The diagram has significant issues: (1) Too
                                  questionable, (5) The diagram doesn't clearly show a top-down fl
    overall_score:              3
  ▶ suggestions:                (6)[ "Reduce circular dependencies - consider showing only the p
                                  depend on, not have dependencies on domain components", "Conside
                                  questionable edges like Error Handling -> Models", "The API Laye
                                  high-level layers, another for internal component relationships"
▼ overall_metrics:
    repo_summary_score:         4
    module_summary_avg:         4.52
    function_summary_avg:       4.68
    architecture_score:         3
    overall_quality:            4.17
▶ function_sampling:            { target_size: 50, sampled_count: 50, total_functions: 278, … }
```


Figure 12: LLM-as-a-Judge evaluation by Sonnet 4.5

Results and Analysis

- **Quantitative Performance metrics:** CodeChat was run end-to-end on six Python repositories spanning small educational projects to mid-sized open-source libraries. The table below summarizes the AST-extracted scale of each repository and the approximate wall-clock processing time:

Repository	Modules	Functions	Processing Time
test_project	6	35	~ 4-5 minutes
gnn_training-master	5	4	~ 1-2 minutes
requests	20	185	~ 20-30 minutes
httpx	22	220	~ 30-40 minutes
flask	34	281	~ 40-50 minutes
click	31	347	~ 50-60 minutes

Processing time scales close to linearly with the function count rather than with repository size as a whole. The gnn_training-master repository finished in just 1–2 minutes because it has only four functions to summarize and the chunking thresholds are never triggered. test_project, despite still being small, jumps to 4–5 minutes once the function count climbs to 35, every additional function maps directly to one more LLM call.

The flask and click comparison shows that function count, not module count, is the real driver of runtime. The two repositories have nearly identical module counts (34 vs 31), but click has 66 more functions and ends up running roughly 10 minutes longer. The same pattern appears between requests and httpx: both contain around 20 modules, but httpx carries 35 more functions (220 vs 185) and an async-heavy design that produces more chunked summarization calls, pushing its runtime from requests's 20–30 minutes up to 30–40 minutes.

Overall, CodeChat shows acceptable scalability for its target use case. Small repositories (< 50 functions) finish in under five minutes, mid-sized projects (150–300 functions) complete within roughly half an hour, and the largest repository in our test set (click, 347 functions) wraps up inside an hour. The dominant performance bottleneck is the function summarizer, which issues one LLM call per function summarized and accounts for roughly 90% of total wall-clock time on every repository. AST parsing is essentially instant, and the higher-level stages, module summarization, repository overview, and logical grouping, run only a handful of times each and stay flat-cost as the codebase grows.

- **Summary Quality Evaluation:** Claude Sonnet 4.5 evaluated CodeChat's generated summaries across three hierarchical levels, function, module, and repository, using four quality dimensions: Factual Accuracy, Completeness, Clarity, and Technical Depth (functions only) or Usefulness (repository only). Each dimension was rated on a 1–5 scale, with an overall_score computed per summary.

Repository	Repository Summary	Module Summaries (Avg)	Function Summaries (Avg)
test_project	4	4.43	4.75
gnn_training-master	4	3.83	4.71
requests	4	3.76	4.74
httpx	4	4.52	4.68
flask	5	4.34	4.67
click	4.5	4.75	4.54
Average	4.25	4.27	4.68

Function summaries were the highest-rated layer (4.68 average), with test_project (4.75) and requests (4.74) leading the table. Across every repository, the judge was most positive about clarity (averaging ~4.95/5) and factual accuracy (~4.74/5), praising the dual human/technical format for cleanly separating intent from implementation. The weakest dimension was completeness (~4.45/5), where the judge consistently flagged that summaries occasionally

omitted optional parameters, return-value edge cases, or side effects. click scored slightly lower than the other repositories (4.54) because its heavy decorator usage and rich parameter handling produced cases where the summary captured the headline behavior but missed nuances of the decorator contract.

Module summaries averaged 4.27, solid but with more variance than the function layer. click (4.75), httpx (4.52), and test_project (4.43) scored highest, with the judge crediting their well-structured modules and clear separation of concerns. requests (3.76) and gnn_training-master (3.83) ranked lowest. For requests, the judge flagged completeness gaps where the module summary captured the main classes but missed important utility functions and helpers and for gnn_training-master, the small codebase produced summaries the judge described as overly brief because there was little semantic content to expand on.

Repository summaries averaged 4.25, but with a sharper bimodal pattern: flask received a perfect 5.00 (the judge highlighted it as a complete, well-organized account of the framework), click scored 4.50, and the other four repositories each received 4.00. The pattern across the four 4.00s is consistent, every repo summary scored 5/5 on accuracy and clarity but lost a point on completeness, with the judge noting that important architectural details (specific transports for httpx, certain auth modules for requests, dataset preparation for gnn_training-master) were not surfaced in the top-level overview. Usefulness also varied (3 for requests, 5 for flask), reflecting the judge's view of how well the repository summary would actually orient a new developer.

Across all three levels, summaries excelled at clarity (averaging ~4.85/5) and factual accuracy (~4.65/5), which is consistent with the design choice of grounding every prompt in AST-extracted facts before asking the LLM to write prose. The most consistent weakness was completeness (~4.05/5), reflecting the line-cap design constraint: human summaries are bounded at 5–6 lines for functions, 6–8 lines for modules, and 8–10 lines for repositories, so when a codebase has a lot of detail to convey, the prompt explicitly forces concision over exhaustiveness. This is a deliberate trade-off, the summaries stay short and readable, but it does mean longer or more sprawling repositories occasionally lose a point on completeness because there is more they could have said.

- **Architecture Diagram Evaluation**: Claude Sonnet 4.5 evaluated CodeChat's generated Mermaid architecture diagrams across three dimensions: Completeness (whether all major modules are represented), Logical Grouping (semantic coherence of the LLM-derived clusters), and Diagram Quality (visual clarity, hierarchy, and absence of clutter).

Repository	Completeness	Logical Grouping	Diagram Quality	Overall
test_project	5	5	3	4
gnn_training-master	5	3	3	3
requests	5	5	3	4
httpx	5	5	2	3

flask	5	5	2	3
click	5	5	2	4
Average	5	4.67	2.5	3.5

Completeness scored a perfect 5.0 across every repository, confirming that the AST Parser's file-discovery logic and the Module Grouper's quota validation reliably surface every major file in the diagram.

Logical Grouping averaged 4.67, with five repositories at 5.0. The judge praised semantic separations such as requests's split into Core HTTP Models / HTTP Communication / Authentication and httpx's clear Transport Layer. Only gnn_training-master scored lower (3.0): with just five modules, the LLM didn't have enough material to form meaningful clusters and produced overly granular groups instead.

Diagram Quality was the weakest dimension at 2.50. The judge consistently flagged cyclic dependencies between groups (e.g., Core ↔ Application Entry in flask, Application Entry ↔ API Layer in httpx) and edge density on larger repositories (httpx had 36+ edges across the group graph). Several repositories also showed __init__.py artifacts pointing outward from the Package Initialization group, which is architecturally inverted.

The diagram-quality scores reflect a specific limitation of the rendering pass. The Module Grouper produces strong, semantically meaningful groups that the judge consistently praises, and the Diagram Generator uses these groups as the boxes in the Mermaid diagram while pulling AST-derived imports for the arrows. However, when files in different groups import each other bidirectionally (a common Python pattern), the diagram renders these as visual cycles because no cycle resolution or edge pruning is applied. Improving the diagram-quality scores will require post-processing the rendered graph, for example, detecting and breaking cycles, separating imports from uses, and excluding documentation or example modules from the runtime view, rather than changing the underlying grouping logic.

- **Overall Results:** CodeChat achieved an overall quality score of 4.26/5.0 averaged across all six repositories, with function summaries scoring highest (4.68) and architecture diagrams scoring lowest (3.50). Individual repository overall scores fell within a narrow band of 4.01 to 4.48, indicating consistent behaviour across very different codebases. click led the table at 4.48 thanks to strong module summaries (4.75) and a clean logical grouping, while gnn_training-master finished at the bottom (4.01), its small five-module size gave both the Module Summarizer and the Module Grouper less semantic content to work with, which dragged its architecture score to 3.0. flask was the only repository to receive a perfect 5.0 on its repository summary, with the judge describing the overview as a complete and well-organized account of the framework.

  The most consistent pattern in the data is the gap between the textual layers (functions, modules, repos, all averaging above 4.25) and the architecture layer (3.50). The bottom-up summarization design produces reliably accurate, clear documentation across every project, while the diagram rendering pass loses points on larger repositories due to unresolved cycles and edge density. The consistency across repositories spanning five to thirty-four modules and four to three-hundred-and-forty-seven functions demonstrates that the hybrid AST + LLM approach

scales cleanly, and every repository exceeded 4.0 overall, validating the core design while clearly identifying the diagram rendering pass as the most actionable area for future work.

Discussion

- **Strengths**
  - **Scalable hierarchical summarization:** The system processed repositories ranging from 4 to 347 functions while maintaining consistent overall quality between 4.01 and 4.48. Function summaries averaged 4.68/5.0 across every repository, showing that the bottom-up aggregation strategy and chunking thresholds (60-line function chunks, 30-function module batches, 40-module repository batches) preserve quality at scale.
  - **Strong semantic grouping without retrieval infrastructure**: The Module Grouper scored 4.67/5.0 on Logical Grouping, with five of six repositories receiving a perfect 5.0, and notably, this is achieved without embeddings, vector stores, or RAG pipelines. The chat interface and grouping step both operate directly on the pre-generated context_<repo>.json, keeping the system simple to deploy and reproduce.
  - **Reproducible evaluation methodology that surfaces real signal:** The LLM-as-a-Judge framework with Sonnet 4.5 judging Haiku 4.5 produces structured JSON scores across four dimensions plus qualitative notes, with a fixed random seed in the stratified function sampling. The sampling itself also strengthened the signal, trivially small functions (like one-line getters or property accessors) tend to attract uniformly high scores regardless of summary quality, which would otherwise mask the weaker scores that genuinely complex functions receive. By skipping dunder methods and sub-five-line stubs and weighting longer functions preferentially, the sampler concentrated the evaluation on the cases where summary quality actually varies, giving every run a reproducible, defensible signal that can be re-applied as the system evolves.
- **Limitations**
  - **Single-judge evaluation:** Quality scoring relies on a single Claude Sonnet 4.5 judge, which shares training lineage with the Haiku 4.5 generator and may bias scores upward. Human expert evaluation would provide more robust validation but at significantly higher cost.
  - **Python-only implementation:** The AST Parser is tightly coupled to Python's ast module and Python-specific syntax. Extending CodeChat to other languages would require language-specific parsers and adapted chunking heuristics, with dynamically-typed languages like JavaScript needing additional handling for their import semantics.
  - **Diagram rendering limits:** The Diagram Generator does no cycle resolution or edge pruning, so larger repositories accumulate visual clutter from bidirectional imports and __init__.py artifacts, reflected in the 2.50/5.0 average diagram-quality score.

- **Challenges**

    - **Transient API errors:** Longer runs hit intermittent 503: ServiceUnavailable and 429: No deployments available errors as Bedrock instances cycled in and out of cooldown, which forced repeated re-runs of partially completed evaluations as seen in Figure 13. The fix was to inspect response bodies and classify them: transient errors trigger exponential backoff retry, while structural errors halt the run so failures aren't masked as success.

```
    src/flask/ctx.py::after_this_request
    src/flask/debughelpers.py::explain_template_loading_attempts
    src/flask/helpers.py::send_file
    src/flask/helpers.py::url_for
    src/flask/helpers.py::stream_with_context
    src/flask/json/__init__.py::jsonify
    src/flask/json/provider.py::dump
    src/flask/json/tag.py::tag
    src/flask/logging.py::wsgi_errors_stream
    src/flask/sansio/app.py::trap_http_exception
    src/flask/sansio/app.py::add_template_test
    src/flask/sansio/app.py::_find_error_handler
    src/flask/sansio/app.py::add_template_global
    src/flask/sansio/app.py::template_filter
    src/flask/sansio/blueprints.py::extend
    JSON parse error: Expecting value: line 1 column 1 (char 0). Trying to fix escapes
    Still failed: Expecting value: line 1 column 1 (char 0)
    Response (first 500 chars): 🚫 **Unexpected Error**

An unexpected error occurred: 429: 🚫 **None**

No deployments available for selected model, Try again in 5 seconds. Passed model=Claude Sonnet 4.5. pre-call-checks=False, cooldown_list=['52993604-1b06-4943-8f68-968279bb45e8', 'f423fe12-4413-4972-8e2
5-184e7ca6e61f']
```

Figure 13: 429 deployment error during evaluation

    - **Working within a strict API budget:** A $5/day TAMU spend cap forced several trade-offs: switching the generator from Sonnet 4.5 to Haiku 4.5, the judge from Opus 4.5 to Sonnet 4.5, and capping function-summary evaluation at ~30 sampled functions per repository rather than full coverage.

    - **Writing just enough prompt to avoid LLM hallucinations:** Early prompts produced fabricated classes and invented design patterns. Fixing this required short grounding instructions ("ground every claim in AST facts", "omit anything unclear") without spoon-feeding the model where over-specified prompts started forcing the model into rigid templates instead of letting it actually reason about the code, so the right balance was guidance, not dictation.

- **Future Work**

    - **Scaling to larger codebases:** With a higher API budget, CodeChat could be applied to repositories beyond the current ~350-function range and stratified sampling could be increased for function evaluation.

    - **Multi-language support:** Swapping Python's ast module for a polyglot parser like tree-sitter would let CodeChat handle JavaScript, Go, Rust, and other languages.

    - **Interactive refinement:** Adding thumbs-up/down feedback on summaries in the Streamlit UI would create a signal for improving prompt design and capturing project-specific conventions over time.

Conclusion

CodeChat shows that pairing AST analysis with LLM semantic reasoning can produce accurate, multi-level repository documentation alongside an interactive interface for exploring unfamiliar codebases. Tested on six Python repositories ranging from 4 to 347 functions, the system achieved an overall quality

score of 4.26/5.0, with function summaries averaging 4.68/5.0 and Logical Grouping scoring 4.67/5.0 across consistently varied projects.

The strongest finding is that grounding every prompt in AST-extracted facts such as function signatures, class hierarchies, imports, and call relationships, keeps the generated summaries factually reliable while still preserving the semantic richness only an LLM can provide. The hierarchical summarization design scaled cleanly through strategic chunking, the Module Grouper produced strong semantic clusters that reflect architectural intent rather than directory layout, and the chat interface answered developer questions directly from the pre-generated artifacts without any embedding or retrieval infrastructure. Processing time scaled close to linearly with function count, small repositories completed in a few minutes and ~350-function projects within an hour, far below the manual effort onboarding documentation typically requires.

Beyond the quantitative results, CodeChat illustrates a broader point: deterministic structural analysis and large language models complement each other naturally. AST parsing provides the factual scaffolding that prevents hallucination, while the LLM provides the semantic interpretation that makes the documentation actually useful to read. Pairing the two through a simple, file-based pipeline turns out to be a practical and scalable foundation for repository understanding, pointing toward a future where high-quality documentation is generated as a routine part of the development workflow rather than a periodic, manual effort.

Code Setup and Execution

- **System Requirements**
    - Python: 3.10 or higher (project tested on 3.13)
    - Operating System: macOS, Linux, or Windows
    - API Access: TAMU AI Chat API key with access to Claude Haiku 4.5 (generation) and Claude Sonnet 4.5 (evaluation)
- **Installation**
    - Clone the repository
        - Run the command: git clone https://github.com/navyapriyanandi-tamu/Directed_Studies_136004813.git
        - cd Directed_Studies_136004813
    - Create and activate a virtual environment
        - python3 -m venv llm
        - source llm/bin/activate
    - Install dependencies
        - pip install -r requirements.txt
- **Configuration**
    - Set the TAMU API key for the current shell session
        - export TAMUS_AI_CHAT_API_KEY=<your_tamu_api_key_here>
    - The key persists only for the current terminal session. To make it persistent, add it to a local .env file in the project root (which python-dotenv loads automatically)

- **Running the Analysis Pipeline**

    - Full pipeline (analysis + Streamlit dashboard) on a target repository, run the command: python3 codechat.py /path/to/target/repository
    - Analysis only (writes context_<repo>.json without launching the dashboard), run the command: python3 analyze_any_repo.py /path/to/target/repository
    - Dashboard only, using an existing context file, run the command: python3 codechat.py /path/to/target/repository --skip-analysis
    - Dashboard without specifying a repo (uses the sidebar dropdown): streamlit run streamlit_app.py
    - The dashboard runs at http://localhost:8501

- **Running Quality Evaluation**

    - Evaluate generated outputs with Claude Sonnet 4.5 as judge, run the command: python3 evaluate_with_sonnet.py <repo_name>

- **Generated Outputs**

    - Core Analysis
        - context_<repo>.json: Full analysis state (AST results, function summaries, module summaries, repository summary, logical groups)
    - Architecture Visualizations (under diagrams/)
        - mermaid_<repo>.mmd: Mermaid.js architecture diagram (color-coded, group-level)
        - dependencies_<repo>.txt: ASCII dependency tree
    - Evaluation
        - evaluation_<repo>.json: LLM-as-a-Judge scores (per-criterion ratings, qualitative notes, function sampling provenance, aggregated overall metrics)

Learning

- **Bottom-up grounding beats brute context:** Asking the LLM to summarize a whole module from raw source produced shallow, generic descriptions whereas feeding it pre-generated function summaries plus AST metadata produced specific, accurate ones. Each layer is only as good as the layer beneath it, and AST grounding is what makes the hierarchy reliable.

- **Prompts need explicit length caps to stay concise:** Without hard limits, the LLM produced sprawling paragraphs that buried the useful information. Adding explicit caps to every prompt - 5–6 lines for functions, 6–8 for modules, 8–10 for repositories, kept output tight and consistent.

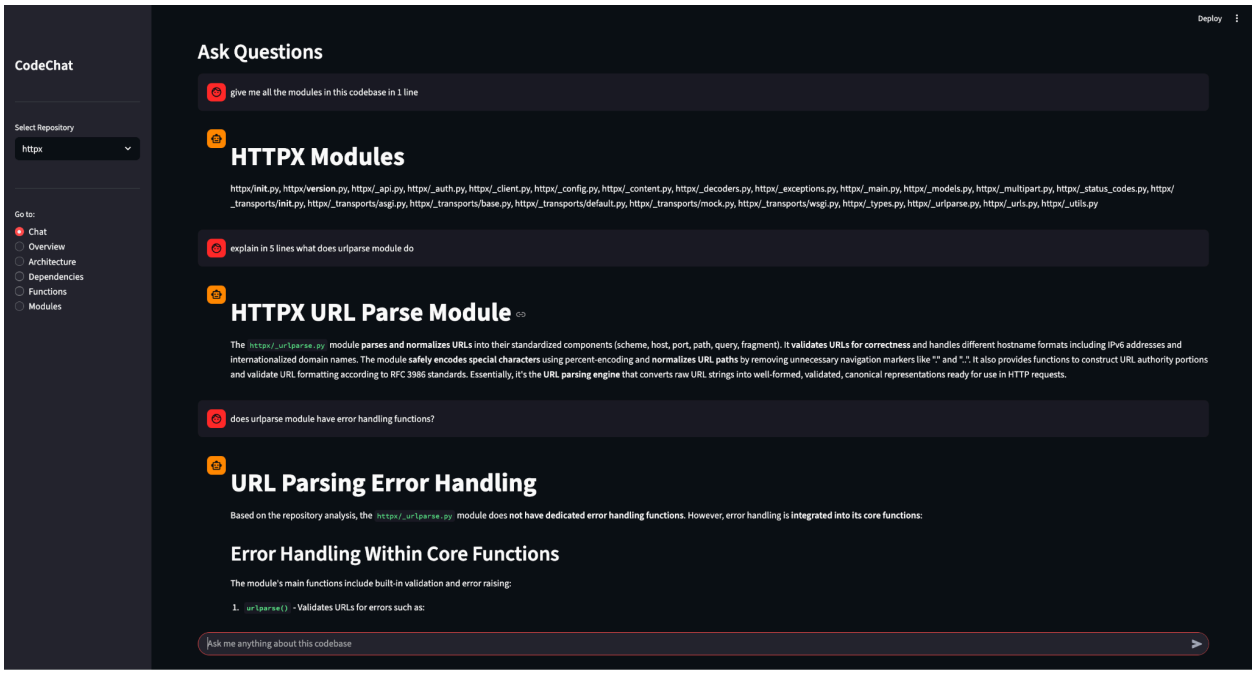
- **Generating a valid Mermaid is harder than it looks:** LLM-generated diagram code frequently fails to parse - empty subgraph titles, special characters in labels, parentheses in node text. Reliably producing renderable diagrams meant moving Mermaid generation out of the LLM and into deterministic code, with explicit sanitization of every label.

References

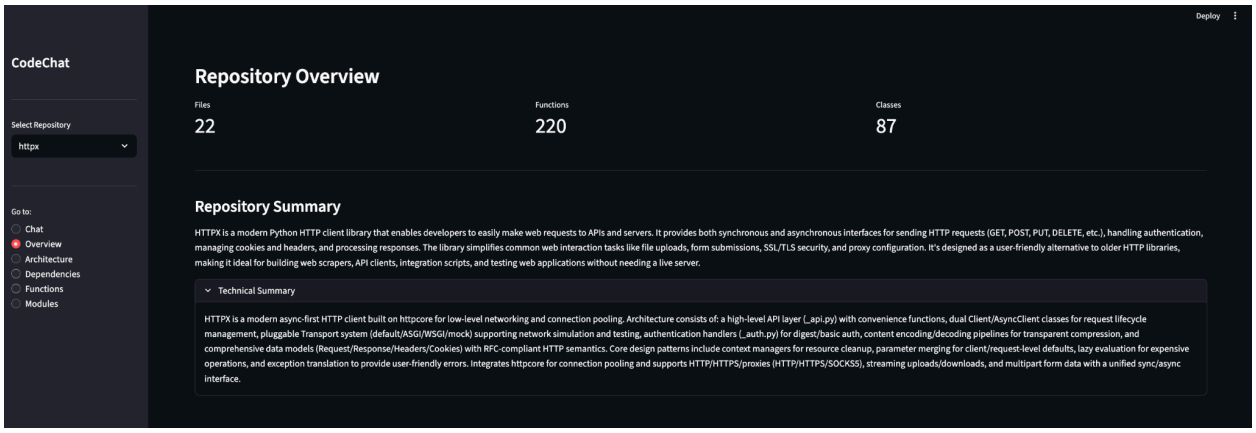
- Python Documentation,"ast - Abstract Syntax Trees," https://docs.python.org/3/library/ast.html

- Y . Luo et al.,"RepoAgent: An LLM-Powered Open-Source Framework for Repository-level Code
- Documentation Generation" 2024, https://arxiv.org/abs/2402.16667
- P. Diggs et al.,"Leveraging LLMs for Legacy Code Modernization: Challenges and Opportunities for LLM-Generated Documentation" 2024, https://arxiv.org/html/2411.14971v1
- M. Li et al.,"Enhancing Static Analysis for Practical Bug Detection: An LLM-Integrated Approach" ACM Transactions on Software Engineering and Methodology, 2024, https://dl.acm.org/doi/10.1145/3649828
- M. Li et al.,"IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities" 2025, https://arxiv.org/abs/2405.17238

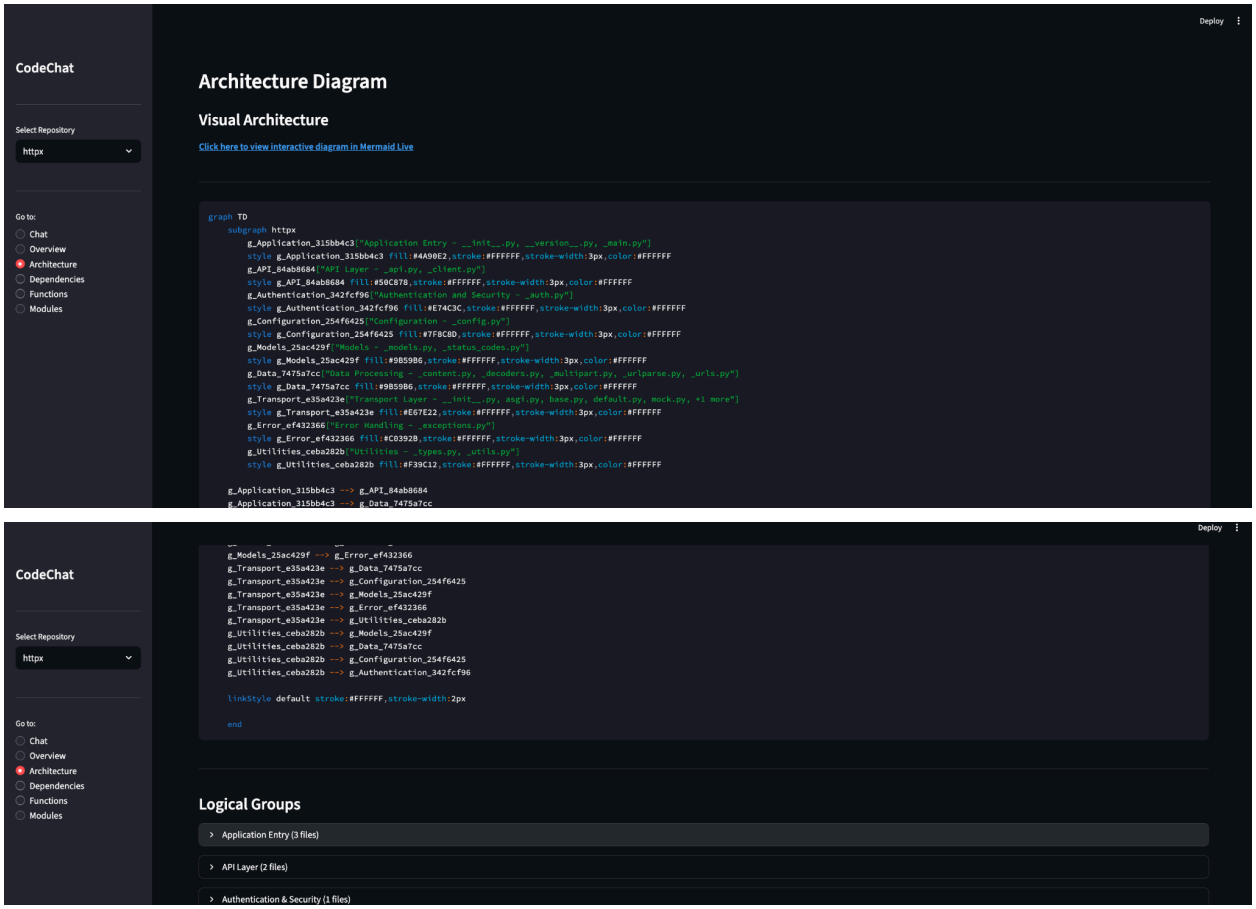
Screenshots of UI



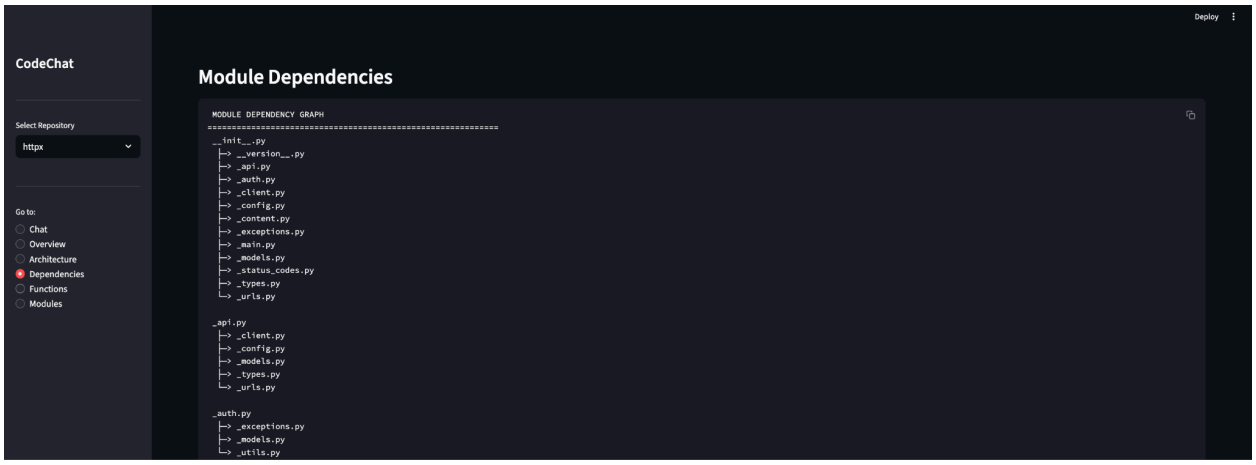
Screen 1: Conversational Querying window



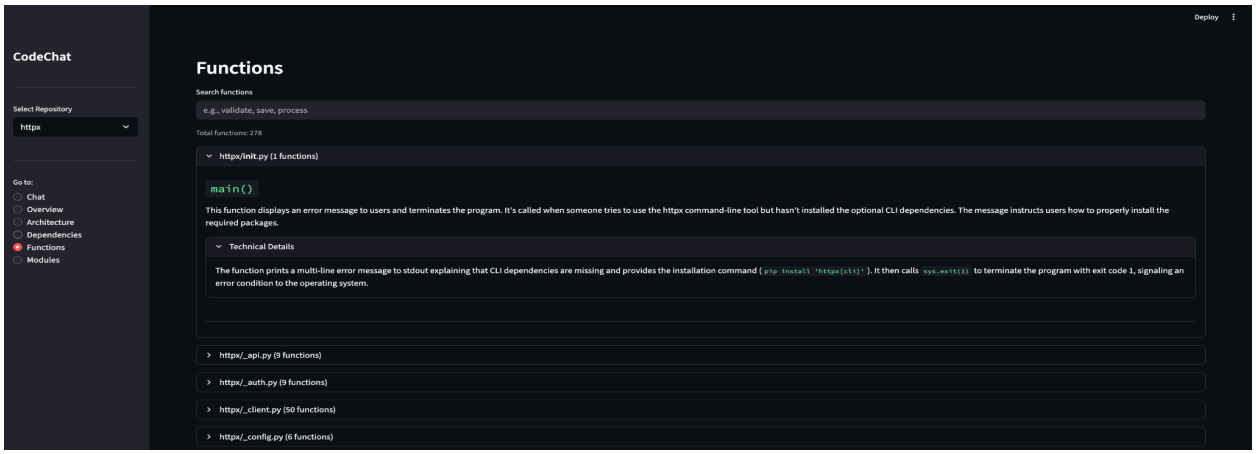
Screen 2: Repository Summary



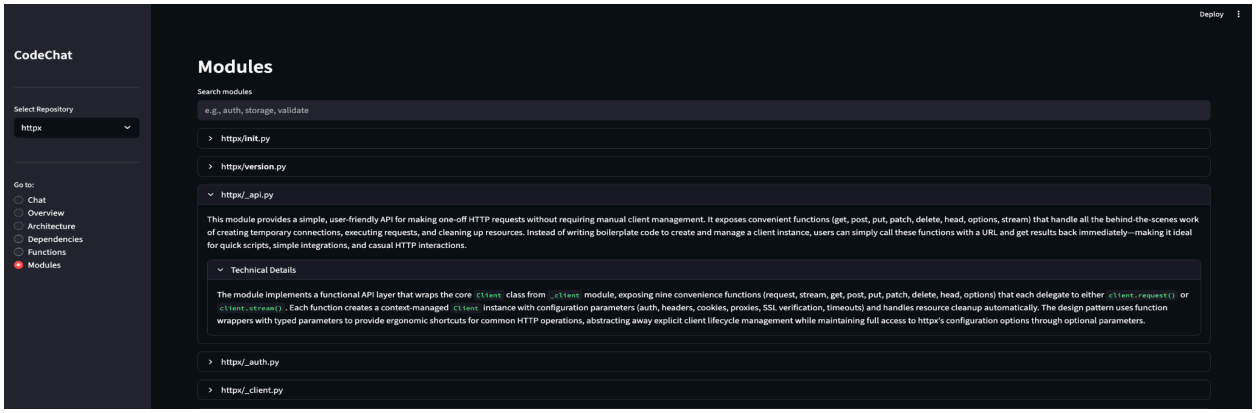
Screen 3: Architecture Visualization with logical groups



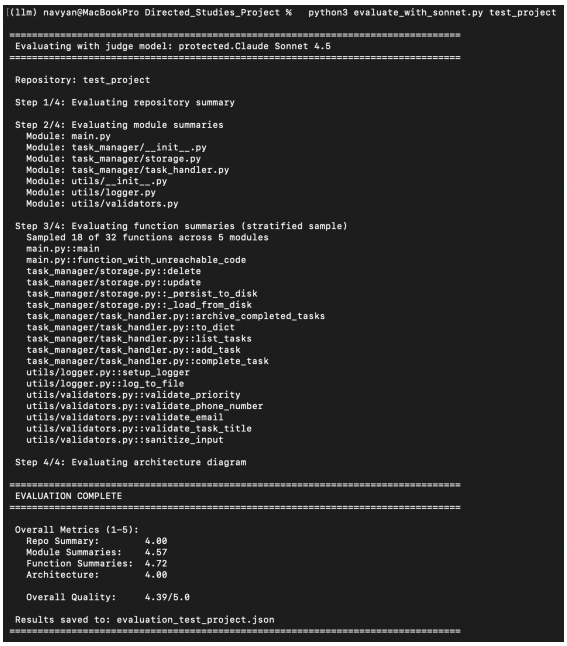
Screen 4: ASCII Dependency Tree



Screen 5: Functions Summary



Screen 6: Module Summary



```
(llm) navyan@MacBookPro Directed_Studies_Project %   python3 evaluate_with_sonnet.py test_project

 ================================================================================
  Evaluating with judge model: protected.Claude Sonnet 4.5
 ================================================================================

  Repository: test_project

  Step 1/4: Evaluating repository summary

  Step 2/4: Evaluating module summaries
    Module: main.py
    Module: task_manager/__init__.py
    Module: task_manager/storage.py
    Module: task_manager/task_handler.py
    Module: utils/__init__.py
    Module: utils/logger.py
    Module: utils/validators.py

  Step 3/4: Evaluating function summaries (stratified sample)
    Sampled 18 of 32 functions across 5 modules
    main.py::main
    main.py::function_with_unreachable_code
    task_manager/storage.py::delete
    task_manager/storage.py::update
    task_manager/storage.py::_persist_to_disk
    task_manager/storage.py::_load_from_disk
    task_manager/task_handler.py::archive_completed_tasks
    task_manager/task_handler.py::to_dict
    task_manager/task_handler.py::list_tasks
    task_manager/task_handler.py::add_task
    task_manager/task_handler.py::complete_task
    utils/logger.py::setup_logger
    utils/logger.py::log_to_file
    utils/validators.py::validate_priority
    utils/validators.py::validate_phone_number
    utils/validators.py::validate_email
    utils/validators.py::validate_task_title
    utils/validators.py::sanitize_input

  Step 4/4: Evaluating architecture diagram

 ================================================================================
  EVALUATION COMPLETE
 ================================================================================

  Overall Metrics (1-5):
    Repo Summary:       4.00
    Module Summaries:   4.57
    Function Summaries: 4.72
    Architecture:       4.00

    Overall Quality:    4.39/5.0

  Results saved to: evaluation_test_project.json
 ================================================================================
```

Screen 7: LLM-as-a-judge Sonnet Evaluation